

SECURITY PROOF OF CONCEPT REPORT

Dhruva

Web Application Security Assessment

Executive Summary

This security assessment was conducted on the target web application (ijsser.org) to identify potential vulnerabilities and security misconfigurations. The assessment employed both automated scanning tools and manual verification techniques to ensure comprehensive coverage. Multiple critical and high-severity vulnerabilities were discovered that require immediate remediation.

A total of 10+ security issues were identified across different categories including infrastructure vulnerabilities, web application flaws, and insecure configurations.

Assessment Scope

Target	Details
Target Host	ijsser.org (162.215.252.35)
Assessment Date	May 28-29, 2026
Tools Used	Nmap, Metasploit, Burp Suite, Custom Scripts

Vulnerability Summary

Severity	Count	Status	Risk Level
Critical	2	Open	Urgent
High	5	Open	High
Medium	3	Open	Medium

Critical Vulnerabilities

1. SQL Injection (CVE-2019-9193)

Severity: CRITICAL

Description: SQL injection vulnerabilities were identified in multiple PHP files including /more2018.php, /indexing-services.php, and others. These allow attackers to execute arbitrary SQL queries against the database.

Impact: An attacker can bypass authentication, extract sensitive data from the database, modify or delete records, and potentially gain complete control over the application and underlying system.

Affected URLs: <http://ijsser.org/more2018.php>, <http://ijsser.org/indexing-services.php>

Remediation: Implement parameterized queries (prepared statements) for all database operations. Use ORM frameworks when possible and apply input validation on all user inputs.

2. Insecure Transport Protocol (HTTP Over Unencrypted Connection)

Severity: CRITICAL

Description: Strict Transport Security (HSTS) is not enforced. The application allows HTTP connections and does not automatically redirect users to HTTPS. Multiple pages are accessible over unencrypted HTTP.

Impact: An attacker positioned on the network (Man-in-the-Middle) can intercept and modify user traffic, steal session cookies, credentials, and sensitive data. This is particularly dangerous on public Wi-Fi networks.

Proof: The application fails to prevent users from connecting via HTTP, and the response headers do not include HSTS directives.

Remediation: Implement HTTP Strict-Transport-Security (HSTS) header with a long max-age (minimum 31536000 seconds). Redirect all HTTP traffic to HTTPS. Obtain a valid SSL/TLS certificate and ensure all resources are served over HTTPS.

High-Severity Vulnerabilities

3. Slowloris Denial of Service (CVE-2007-6750)

Severity: HIGH

Description: The web server is vulnerable to Slowloris attacks. This vulnerability allows an attacker to keep many connections to the server open by sending partial HTTP requests, gradually starving the server of resources.

Technical Details: Slowloris attempts to keep many connections to the target web server open by sending partial requests. It accomplishes this by opening connections to the target web server and sending a partial request, keeping the connection open as long as possible by periodically sending additional partial headers, but never completing the request.

Impact: Service availability is compromised. With minimal bandwidth requirements, an attacker can cause a complete Denial of Service against the web application.

Remediation: Configure timeouts for incomplete requests. Use a reverse proxy with connection limits. Implement rate limiting per IP address. Deploy a Web Application Firewall (WAF) with Slowloris protection.

4. Cross-Domain Referrer Leakage

Severity: HIGH

Description: The application does not implement proper referrer policies. When users navigate away from the site, the URL containing sensitive parameters or session information is sent as the Referer header to third-party sites.

Impact: Sensitive information including user IDs, session tokens, and other parameters may be leaked to third-party websites through the Referer header, potentially compromising user privacy and security.

Remediation: Implement the Referrer-Policy header with a value of 'no-referrer' or 'strict-origin-when-cross-origin' to control referrer information leakage.

5. Exposed Backup Files

Severity: HIGH

Description: Backup files with common extensions (.bak, .old, .backup, .sql) are accessible through the web server. These files contain sensitive source code and configuration information.

Impact: Source code disclosure enables attackers to identify additional vulnerabilities, understand application logic, and potentially extract hardcoded credentials or API keys.

Remediation: Remove all backup files from the web root. Configure the web server to deny access to backup file extensions using .htaccess or web server configuration. Implement proper file management procedures.

6. Email Address Disclosure

Severity: HIGH

Description: Multiple email addresses are exposed in publicly accessible pages and metadata, including administrative and staff contact information.

Impact: Exposed email addresses can be used for phishing attacks, social engineering, and spam campaigns targeting employees and users.

Remediation: Remove email addresses from publicly accessible pages. Use email obfuscation techniques if contact information must be displayed. Implement JavaScript-based email protection or use contact forms instead of direct email links.

7. DOM-Based Denial of Service

Severity: HIGH

Description: The application contains DOM-based denial of service vulnerabilities where specially crafted inputs can cause excessive client-side processing or DOM operations, crashing the browser tab.

Impact: Users experience browser crashes or hangs, resulting in poor user experience and potential session loss. This can be weaponized against specific users.

Remediation: Validate and sanitize all user inputs before DOM manipulation. Implement performance monitoring to detect excessive DOM operations. Use virtual DOM frameworks that optimize rendering.

Medium-Severity Vulnerabilities

8. Clickjacking Vulnerability

Severity: MEDIUM

Description: The application does not implement X-Frame-Options header, allowing the page to be framed by other websites. This enables clickjacking attacks.

Impact: Attackers can overlay transparent iframes over legitimate page elements, tricking users into performing unintended actions such as transferring funds or changing account settings.

Remediation: Implement X-Frame-Options header set to 'DENY' or 'SAMEORIGIN'. Use Content-Security-Policy frame-ancestors directive as a modern alternative.

9. Cacheable HTTPS Response

Severity: MEDIUM

Description: Sensitive HTTPS responses are marked as cacheable, allowing sensitive information to be stored in browser cache or CDN caches.

Impact: Sensitive user data may be retrieved from cache by other users sharing the same computer or through cache poisoning attacks.

Remediation: Set Cache-Control headers to 'no-cache, no-store, must-revalidate' for sensitive pages. Use 'private' directive for user-specific content.

10. Cross-Domain Script Include

Severity: MEDIUM

Description: The application includes external JavaScript files from untrusted or third-party domains without proper validation or integrity checks.

Impact: If third-party domains are compromised, malicious code can be injected into the application, potentially compromising all users.

Remediation: Use Subresource Integrity (SRI) hashes to verify external script integrity. Whitelist trusted CDNs. Implement Content Security Policy (CSP) to restrict script sources.

Infrastructure Findings

Target IP: 162.215.252.35

Open Ports Detected

Port	Service	Risk Level
21/tcp	FTP	HIGH
22/tcp	SSH	MEDIUM
80/tcp	HTTP	HIGH
443/tcp	HTTPS	MEDIUM
465/tcp	SMTPS	MEDIUM
3306/tcp	MySQL	CRITICAL

The exposure of MySQL database service (port 3306) on the public internet is extremely dangerous and should be immediately restricted to only authorized internal IP addresses.

Recommendations & Remediation Plan

Immediate Actions (Within 24-48 Hours)

- Block external access to MySQL port (3306). Use firewall rules to restrict database access to application servers only.
- Implement HTTPS redirection and HSTS headers on all web servers.
- Remove all backup files from the web root and implement .htaccess rules to deny access to backup file extensions.

Short-Term Fixes (1-2 Weeks)

- Conduct code review of all PHP applications, specifically /more2018.php and /indexing-services.php, to identify and remediate SQL injection vulnerabilities.
- Implement prepared statements and parameterized queries throughout the application.
- Configure Web Application Firewall (WAF) with rules to detect and block Slowloris attacks and SQL injection attempts.
- Implement security headers: X-Frame-Options, Content-Security-Policy, Referrer-Policy, and Cache-Control.

Long-Term Improvements (1-3 Months)

- Implement a Web Application Firewall (WAF) solution such as ModSecurity or AWS WAF.
- Conduct comprehensive security code review and implement secure coding practices training for development team.
- Migrate to modern frameworks with built-in security features (Laravel, Spring Boot, Django, etc.).
- Implement continuous security testing and vulnerability scanning as part of CI/CD pipeline.

Conclusion

The security assessment has identified multiple critical and high-severity vulnerabilities that pose significant risks to the application, its users, and the underlying infrastructure. The most pressing concern is the exposure of sensitive services and the presence of SQL injection vulnerabilities that could lead to complete system compromise.

Immediate remediation of critical issues is essential. We recommend prioritizing the closure of exposed database ports, implementation of HTTPS enforcement, and remediation of SQL injection vulnerabilities. A phased approach should be taken for medium and low-severity issues over the next 3 months.

With proper implementation of the recommendations outlined in this report, the security posture of the application can be significantly improved, protecting both users and organizational assets.